

EKS Ingress Setup — Full End-to-End

This example explains:

- What we are building
- Why each component is needed
- Full YAML files
- AWS concepts
- Kubernetes concepts
- Traffic flow
- Verification
- Troubleshooting

Goal:

Deploy an NGINX application in Amazon EKS and expose it to the internet using:

- AWS Application Load Balancer (ALB)
- Kubernetes Ingress
- AWS Load Balancer Controller

1. What We Are Building

We will build this architecture:

User Browser



AWS ALB (Public Load Balancer)



Kubernetes Ingress



Kubernetes Service



NGINX Pods

2. Understanding Each Component

Component 1 — Pod

A Pod is the smallest unit in Kubernetes.

Think:

Pod = Running Container

Example:

NGINX container running inside a pod

Component 2 — Deployment

Deployment manages pods automatically.

Why needed?

Because pods can:

- Crash
- Restart
- Get deleted

Deployment ensures:

- Desired number of pods always running

Example:

replicas: 2

Means:

Always keep 2 pods running

Component 3 — Service

Pods get random IP addresses.

Problem:

Pod IP changes after restart

Service provides:

- Stable internal IP
- Stable DNS name

Example:

nginx-service.default.svc.cluster.local

Component 4 — Ingress

Ingress exposes applications externally.

Without Ingress:

- Every app needs separate LoadBalancer Service
- Expensive
- Hard to manage

Ingress allows:

- One ALB
 - Multiple applications
 - Centralized routing
-

Component 5 — AWS Load Balancer Controller

Very important.

Ingress YAML itself cannot create AWS ALB.

Controller:

- Reads ingress YAML
- Calls AWS APIs

- Creates ALB automatically

Without controller:

Ingress does nothing

3. Environment Used

Example values:

Component	Value
Cluster Name	demo-eks
Region	us-east-1
Namespace	default
App	nginx
VPC ID	vpc-123456
AWS Account ID 123456789012	

4. Prerequisites

Install:

Install kubectl

Why?

Used to communicate with Kubernetes cluster.

Verify:

`kubectl version --client`

Install AWS CLI

Why?

Used to communicate with AWS.

Verify:

```
aws --version
```

Install eksctl

Why?

Simplifies EKS management.

Verify:

```
eksctl version
```

Install Helm

Why?

Used to install Kubernetes packages.

Verify:

```
helm version
```

5. Verify EKS Cluster

Run:

```
kubectl get nodes
```

Expected:

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-1-10.ec2.internal	Ready	<none>	1d	v1.30

Why This Step?

Confirms:

- Cluster reachable

- Worker nodes healthy
 - Kubernetes API working
-

6. Understand OIDC and IRSA

This is the MOST IMPORTANT concept.

Problem Without IRSA

Controller pod needs AWS permissions.

Bad approach:

Store AWS keys inside pod

Problems:

- Security risk
 - Hard to manage
-

Better Approach — IRSA

IRSA:

IAM Roles for Service Accounts

Flow:

Kubernetes Service Account

↓

Assumes IAM Role

↓

Gets temporary AWS credentials

Secure and production standard.

7. Associate OIDC Provider

Run:

```
eksctl utils associate-iam-oidc-provider \
--region us-east-1 \
--cluster demo-eks \
--approve
```

Why?

Creates trust between:

- EKS cluster
- AWS IAM

Without this:

Pods cannot assume IAM roles

8. Download IAM Policy

Run:

```
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json
```

Why?

Policy contains permissions like:

- Create ALB
 - Create target groups
 - Modify security groups
 - Register targets
-

9. Create IAM Policy

Run:

```
aws iam create-policy \  
--policy-name AWSLoadBalancerControllerIAMPolicy \  
--policy-document file://iam_policy.json
```

Why?

Creates reusable IAM policy in AWS.

Without permissions:

Controller gets AccessDenied errors

10. Create IAM Service Account

Run:

```
eksctl create iamserviceaccount \  
--cluster=demo-eks \  
--namespace=kube-system \  
--name=aws-load-balancer-controller \  
--role-name AmazonEKSLoadBalancerControllerRole \  
--attach-policy-  
arn=arn:aws:iam::123456789012:policy/AWSLoadBalancerControllerIAMPolicy \  
--approve
```

What Happens Internally?

Creates:

Resource	Purpose
----------	---------

Resource	Purpose
IAM Role	AWS permissions
Service Account	Kubernetes identity
Trust Policy	Connects Kubernetes to AWS

11. Add Helm Repository

Run:

```
helm repo add eks https://aws.github.io/eks-charts
```

Update:

```
helm repo update
```

Why?

Helm repo contains:

- AWS Load Balancer Controller package

Similar to:

apt repository

12. Install AWS Load Balancer Controller

Run:

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
```

```
-n kube-system \
```

```
--set clusterName=demo-eks \
```

```
--set serviceAccount.create=false \
```

```
--set serviceAccount.name=aws-load-balancer-controller \
```

```
--set region=us-east-1 \
```

```
--set vpcId=vpc-123456
```

Explain Each Parameter

clusterName

```
--set clusterName=demo-eks
```

Why?

Controller must know which cluster to manage.

serviceAccount.create=false

Why?

We already created secure IRSA-enabled service account.

serviceAccount.name

Links controller pod to IAM-enabled service account.

region

Tells controller:

Which AWS region to create resources in

vpcId

ALB must be created inside correct VPC.

13. Verify Controller

Run:

```
kubectl get pods -n kube-system
```

Expected:

aws-load-balancer-controller-xxxxx 1/1 Running

Why?

If controller is not running:

Ingress will not create ALB

14. Create Application Deployment

Create file:

nginx-deployment.yaml

Full Deployment YAML

apiVersion: apps/v1

kind: Deployment

metadata:

name: nginx-deployment

spec:

replicas: 2

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

```
- name: nginx  
  image: nginx:latest  
  ports:  
    - containerPort: 80  
  readinessProbe:  
    httpGet:  
      path: /  
      port: 80  
    initialDelaySeconds: 5  
    periodSeconds: 10
```

Explain Every Section

apiVersion

apiVersion: apps/v1

Which Kubernetes API version to use.

kind

kind: Deployment

Tells Kubernetes:

Create a Deployment object

metadata

metadata:

name: nginx-deployment

Name of deployment.

replicas

replicas: 2

Run 2 pods.

Why?

High availability.

selector

selector:

matchLabels:

app: nginx

Deployment manages pods having:

app=nginx

template

Defines pod configuration.

container image

image: nginx:latest

Downloads NGINX image from Docker Hub.

readinessProbe

Very important in production.

Checks:

Is application ready to receive traffic?

Without this:

- ALB may send traffic too early

- Causes failures

Deploy Application

Run:

```
kubectl apply -f nginx-deployment.yaml
```

Verify Pods

Run:

```
kubectl get pods
```

Expected:

```
nginx-deployment-xxxxx Running
```

15. Create Service

Create:

```
nginx-service.yaml
```

Full Service YAML

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: nginx-service
```

```
spec:
```

```
  selector:
```

```
    app: nginx
```

```
  ports:
```

```
    - protocol: TCP
```

port: 80

targetPort: 80

type: ClusterIP

Explain Service

selector

selector:

app: nginx

Service forwards traffic to:

Pods having app=nginx

port

port: 80

Service listens on port 80.

targetPort

targetPort: 80

Forward traffic to container port 80.

type: ClusterIP

Internal Kubernetes networking only.

Ingress will expose externally.

Apply Service

Run:

```
kubectl apply -f nginx-service.yaml
```

Verify Service

Run:

```
kubectl get svc
```

16. Create Ingress

Create:

```
ingress.yaml
```

Full Ingress YAML

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
```

```
metadata:
```

```
  name: nginx-ingress
```

```
  annotations:
```

```
    kubernetes.io/ingress.class: alb
```

```
    alb.ingress.kubernetes.io/scheme: internet-facing
```

```
    alb.ingress.kubernetes.io/target-type: ip
```

```
spec:
```

```
  rules:
```

```
    - http:
```

```
      paths:
```

```
        - path: /
```


pathType: Prefix

backend:

service:

name: nginx-service

port:

number: 80

Explain Ingress Fully

ingress.class: alb

Tells Kubernetes:

Use AWS ALB Controller

scheme: internet-facing

Creates PUBLIC ALB.

Alternative:

internal

Creates private ALB.

target-type: ip

Registers pod IPs directly.

Benefits:

- Better performance

- Native Kubernetes integration
-

path: /

Route all traffic to:

nginx-service

Apply Ingress

Run:

```
kubectl apply -f ingress.yaml
```

17. Verify Ingress

Run:

```
kubectl get ingress
```

Expected:

NAME	CLASS	HOSTS	ADDRESS
nginx-ingress	<none>	*	k8s-default-xxxx.elb.amazonaws.com

What Happens Internally?

Controller sees ingress.

Then:

1. Creates ALB
2. Creates target group
3. Registers pod IPs
4. Creates listener rules
5. Configures health checks

Automatically.

18. Access Application

Open:

`http://<ALB-DNS>`

Expected:

Welcome to nginx!

19. Understand Full Traffic Flow

Step 1

User opens browser:

`http://alb-dns-name`

Step 2

Traffic reaches ALB.

Step 3

ALB checks ingress rules.

Step 4

Ingress routes request to:

`nginx-service`

Step 5

Service forwards request to pod.

Step 6

Pod returns response.

20. AWS Resources Created Automatically

After ingress creation:

AWS creates:

Resource	Purpose
ALB	Public traffic entry
Target Group	Tracks pod health
Security Groups	Allow traffic
Listeners	Port 80/443
Listener Rules	Routing rules

21. Important Production Concepts

HTTPS Setup

Production should ALWAYS use HTTPS.

Example annotations:

```
alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-east-1:123456789012:certificate/xxxx
```

```
alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
```

```
alb.ingress.kubernetes.io/ssl-redirect: '443'
```

Why SSL Redirect?

Automatically redirects:

HTTP → HTTPS

22. Path-Based Routing Example

/app1 → app1-service

/app2 → app2-service

One ALB hosts multiple apps.

Cost optimization.

23. Host-Based Routing Example

api.company.com

admin.company.com

Different domains routed differently.

24. Common Beginner Problems

Problem 1 — ALB Not Created

Check:

kubectl logs -n kube-system deployment/aws-load-balancer-controller

Usually:

- IAM issue
 - OIDC issue
-

Problem 2 — Ingress ADDRESS Empty

Cause:
Controller failed.

Problem 3 — Target Group Unhealthy

Check:

- Container port
 - Readiness probe
 - Health endpoint
-

Problem 4 — 502 Bad Gateway

Usually:

- App crashed
 - Wrong port mapping
-

25. Verify AWS Console

Go to:

EC2 → Load Balancers

Verify:

- ALB exists
 - Listener exists
 - Security group exists
-

Verify Target Groups

Check:

- Targets healthy

- Pod IPs registered
-

26. Cleanup

Delete ingress:

```
kubectl delete -f ingress.yaml
```

Delete service:

```
kubectl delete -f nginx-service.yaml
```

Delete deployment:

```
kubectl delete -f nginx-deployment.yaml
```

Final Understanding

You now understand:

Component Responsibility

Pod	Runs application
Deployment	Maintains pods
Service	Stable networking
Ingress	External routing
ALB	AWS public load balancer
Controller	Creates AWS resources
IRSA	Secure AWS authentication

Real Production Architecture

Internet

↓

Route53 DNS



AWS WAF



AWS ALB



Ingress



Service



Pods
